



UNIVERSIDADE VEIGA DE ALMEIDA
CURSO DE CIÊNCIA DA COMPUTAÇÃO

CAIO PARADA OLIVEIRA PLANINSCHKEK - 1240205596

CARLOS MATHEUS MARINHO TAVARES - 1240119803

DAVI HASSON CASTRO - 1240117220

EDUARDO DE OLIVEIRA FERNANDES - 1240104170

FELIPE VALENTIM GABRY - 1240114958

GABRIEL ALBUQUERQUE VARELA SANTARELLO - 1240110815

GABRIEL SILVA FRANCISCONE - 1240117399

IGOR ROCHA LOBATO - 1240114118

JOÃO VICTOR BERÇOT CHABUDET CABRAL - 1240108001

LUCAS VASCONCELOS TABOSA DE ALMEIDA - 1250100134

LUIZ FELIPE DE AGUIAR PASSARONI - 1240114559

MARIA FERNANDA LOURENÇO CLASS TEIXEIRA – 1240113501

RELATÓRIO DE PROJETO:

Guia de Streaming — Plot Twist

Rio de Janeiro RJ

2026

CAIO PARADA OLIVEIRA PLANINSCHKE - 1240205596
CARLOS MATHEUS MARINHO TAVARES - 1240119803
DAVI HASSON CASTRO - 1240117220
EDUARDO DE OLIVEIRA FERNANDES - 1240104170
FELIPE VALENTIM GABRY - 1240114958
GABRIEL ALBUQUERQUE VARELA SANTARELLO - 1240110815
GABRIEL SILVA FRANCISCONE - 1240117399
IGOR ROCHA LOBATO - 1240114118
JOÃO VICTOR BERÇOT CHABUDET CABRAL - 1240108001
LUCAS VASCONCELOS TABOSA DE ALMEIDA - 1250100134
LUIZ FELIPE DE AGUIAR PASSARONI - 1240114559
MARIA FERNANDA LOURENÇO CLASS TEIXEIRA – 1240113501

RELATÓRIO DE PROJETO:

Guia de Streaming — Plot Twist

Relatório do projeto desenvolvido para
cumprimento do componente curricular
Laboratório de Desenvolvimento de Software
do Curso de Ciência da Computação da
Universidade Veiga de Almeida.

Professor: Paulo Andrade

Semestre: 2026.1

Rio de Janeiro RJ

2026

RESUMO

Escolher onde assistir a um filme ou série no Brasil exige consultar várias plataformas de streaming, cada uma com seu próprio catálogo e suas próprias regras de assinatura, aluguel e compra. O Plot Twist responde a esse problema com uma aplicação web full-stack que reúne busca, descoberta e ficha de títulos a partir da base da The Movie Database (TMDb), em português e para a região brasileira, e que mostra de forma direta em quais serviços cada título está disponível. O usuário cria conta, busca filmes e séries, abre a ficha completa com sinopse, elenco e provedores, e mantém um histórico pessoal de avaliações, vistos e favoritos. A solução adota arquitetura Backend-for-Frontend (BFF): o navegador conversa apenas com o frontend Next.js, que repassa as requisições à API NestJS de servidor para servidor, mantendo segredos e token de sessão fora do cliente. O desenvolvimento seguiu metodologia ágil baseada em Scrum, com backlog em issues do GitHub, entregas por sprint e integração por Pull Request com revisão de pares, sustentado por testes automatizados e por uma esteira de integração e entrega contínuas. O sistema está publicado e operante: o frontend roda na Vercel em <https://lab-dev-software-front.vercel.app> e a API roda em uma máquina virtual da Azure, com HTTPS, em <https://api-uva.eduoncode.com>.

Palavras-chave: streaming; desenvolvimento web full-stack; arquitetura BFF; REST; metodologia ágil.

Sumário

RESUMO.....	3
1 INTRODUÇÃO.....	4
2 FUNDAMENTAÇÃO TEÓRICA	5
3 DESENVOLVIMENTO DO SISTEMA.....	7
3.1 Arquitetura da aplicação.....	7
3.2 Banco de dados.....	8
3.3 Backend	9
3.4 Frontend.....	10
3.5 API REST	10
4 DOCUMENTAÇÃO DAS TELAS.....	11
4.1 Home	12
4.2 Busca	13
4.3 Ficha do título.....	14
4.4 Login.....	15
4.5 Cadastro	16
4.6 Perfil	17
5 ORGANIZAÇÃO ÁGIL E BACKLOG.....	18
6 VERSIONAMENTO E GITHUB	19
7 RESULTADOS	20
8 CONSIDERAÇÕES FINAIS	21
REFERÊNCIAS	22

1 INTRODUÇÃO

Este relatório documenta o Guia de Streaming, projeto desenvolvido na disciplina Laboratório de Desenvolvimento de Software da Universidade Veiga de Almeida (UVA) ao longo do semestre 2026.1. O trabalho exercita, de ponta a ponta, o ciclo de desenvolvimento de software de um produto real: levantamento de requisitos, modelagem, implementação de backend e frontend, integração com serviço externo, testes, versionamento colaborativo e publicação em ambiente de produção.

O problema que motiva o sistema é cotidiano: no Brasil, decidir onde assistir a um filme ou a uma série exige consultar diversas plataformas de streaming, cada uma com catálogo próprio e regras distintas de assinatura, aluguel e compra. Essa fragmentação torna a escolha demorada e sujeita a erro, especialmente quando o título muda de provedor ou está disponível em mais de uma modalidade.

O objetivo geral do projeto é, portanto, ajudar o usuário a decidir onde assistir a um título. Em vez de abrir cada serviço de streaming separadamente, a pessoa busca o filme ou a série em um único lugar e recebe, na ficha do título, a lista de provedores disponíveis no Brasil, separados por assinatura, aluguel e compra. Em torno dessa funcionalidade central, o produto oferece um histórico pessoal que permite avaliar, marcar como visto e favoritar títulos.

Como objetivos específicos, definiram-se: (a) implementar cadastro e login com sessão segura baseada em JWT e cookie HttpOnly; (b) integrar o catálogo da TMDB em português e para a região brasileira, com cache para reduzir latência e consumo da API externa; (c) construir uma API REST de domínio isolada do navegador por meio do padrão Backend-for-Frontend; (d) desenvolver uma interface responsiva com tema claro e escuro; e (e) publicar o sistema em ambiente de produção com HTTPS e esteira de integração e entrega contínuas.

O escopo do MVP cobre cadastro e login com sessão segura, busca de filmes e séries, ficha do título com a informação de onde assistir, descoberta por gênero e por populares na Home, avaliação de 1 a 10, marcação de visto, favoritos, página de perfil com os totais e as listas do usuário, além de tema claro e escuro e layout responsivo. Ficaram fora de escopo, por decisão de projeto, funcionalidades que ampliariam o produto sem serem essenciais para o problema central: rede social entre usuários, comentários, lista de “quero assistir” (watchlist), motor de recomendação e área administrativa com controle de papéis (RBAC).

O restante do documento está organizado da seguinte forma: a seção 2 apresenta a fundamentação teórica; a seção 3 descreve o desenvolvimento do sistema, cobrindo arquitetura,

banco de dados, backend, frontend e API REST; a seção 4 documenta as telas; a seção 5 relata a organização ágil e o backlog; a seção 6 trata do versionamento e do GitHub; a seção 7 apresenta os resultados; e a seção 8 traz as considerações finais, seguidas das referências.

2 FUNDAMENTAÇÃO TEÓRICA

O Plot Twist se apoia em um conjunto de conceitos consolidados de desenvolvimento web, resumidos a seguir.

Desenvolvimento web full-stack. O sistema é dividido em duas aplicações cooperantes, uma de frontend e uma de backend, cada uma com responsabilidade clara. Essa separação permite que cada camada evolua e seja testada de forma independente, e organiza o trabalho do time por especialidade.

REST. A comunicação com o domínio acontece por uma API REST, estilo arquitetural definido por Fielding (2000). Recursos são identificados por URLs, operações usam os verbos HTTP (GET, POST, DELETE) e o estado é transferido em JSON. Os códigos de status HTTP comunicam o resultado de cada operação, incluindo casos de conflito de regra de negócio.

Arquitetura BFF (Backend-for-Frontend). O navegador não conversa diretamente com a API de domínio. Ele fala apenas com o frontend Next.js, que expõe rotas same-origin em /api/* e atua como intermediário. Esse padrão, descrito por Newman (2015) como variação do estilo de microsserviços, mantém segredos e token de sessão fora do JavaScript do cliente e entrega ao navegador um contrato moldado para a interface, reduzindo a superfície exposta.

ORM (Prisma). O acesso ao banco é feito por um Object-Relational Mapper, que descreve o schema de forma declarativa, gera um cliente tipado e versiona a evolução do banco por migrations. Isso aproxima o modelo de dados do código da aplicação e dá rastreabilidade às mudanças de estrutura.

Autenticação com JWT e cookie HttpOnly. A identidade do usuário é provada por um JSON Web Token (JWT), padrão definido na RFC 7519 (JONES; BRADLEY; SAKIMURA, 2015), com validade de 24 horas. O token trafega para o navegador dentro de um cookie HttpOnly, inacessível a scripts do cliente, o que mitiga o risco de roubo de sessão por código malicioso na página.

Consumo de API externa (TMDB). O catálogo de filmes e séries não é mantido localmente. Ele vem da The Movie Database (TMDB), consultada em português e para a região brasileira. O sistema persiste somente o que é próprio do usuário, e usa cache para reduzir latência e consumo da API externa.

Testes automatizados. A qualidade do software é sustentada por testes que rodam sem intervenção manual: testes unitários verificam regras de negócio isoladas, enquanto testes end-to-end percorrem os fluxos completos da API. No projeto, o backend usa Jest e o frontend usa Vitest, e os testes são executados automaticamente na esteira de integração contínua a cada mudança.

Versionamento e integração contínua. O código é versionado com Git, seguindo o fluxo de feature branches e revisão por Pull Request descrito por Chacon e Straub (2014). As mensagens de commit seguem a convenção Conventional Commits, que padroniza prefixos como feat, fix e refactor e facilita a leitura do histórico. A integração e a entrega contínuas (CI/CD) automatizam build, testes e publicação, reduzindo o risco de erro humano no caminho até a produção.

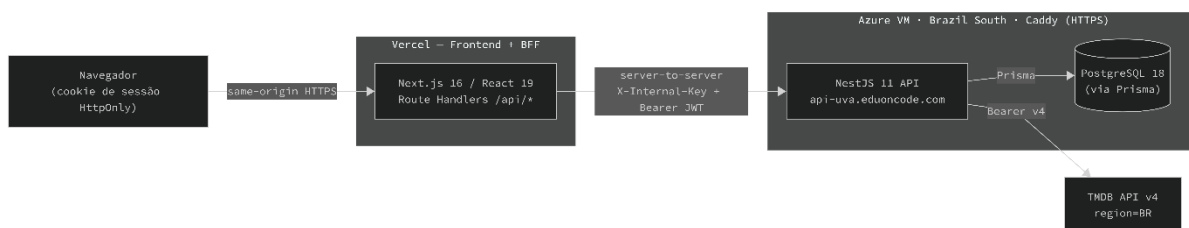
Metodologia ágil e Scrum. O projeto foi conduzido de forma iterativa e incremental, seguindo os princípios do Manifesto Ágil (BECK et al., 2001) e o framework Scrum (SCHWABER; SUTHERLAND, 2020). O backlog viveu em issues do GitHub, organizadas por epic e por sprint, com entregas por incremento demonstrável e revisão por pares antes da integração.

3 DESENVOLVIMENTO DO SISTEMA

Esta seção descreve como o sistema foi construído. A subseção 3.1 apresenta a arquitetura geral e o fluxo da aplicação; a 3.2 detalha o modelo de dados; a 3.3 e a 3.4 cobrem, respectivamente, o backend e o frontend; e a 3.5 documenta o contrato da API REST.

3.1 Arquitetura da aplicação

Figura 1 – Arquitetura da aplicação Plot Twist



Fonte: elaborado pelos autores (2026).

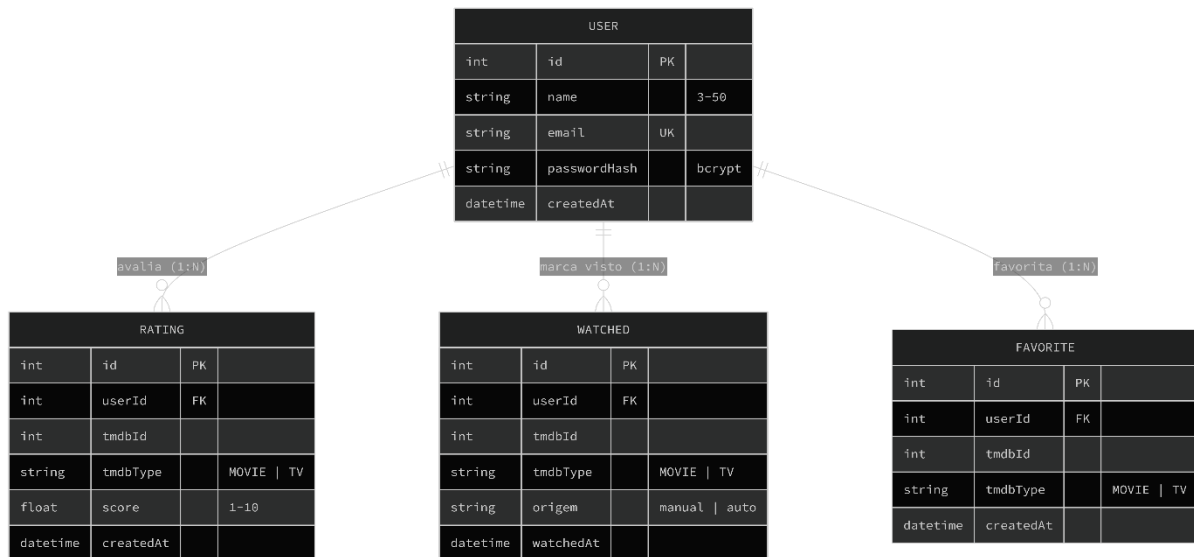
O fluxo segue do navegador para o BFF, do BFF para a API NestJS, e da API para a TMDb e para o PostgreSQL, conforme a Figura 1. O navegador faz chamadas same-origin em HTTPS para as rotas `/api/*` do Next.js. O Next.js renderiza a aplicação React e, no mesmo serviço, atua como BFF por meio de Route Handlers: ele lê o cookie de sessão e repassa a requisição à API NestJS de servidor para servidor, acrescentando o header `X-Internal-Key` e, nas rotas protegidas, o header `Authorization: Bearer <jwt>`. A API consulta a TMDb com `language=pt-BR` e `region=BR` e persiste no PostgreSQL apenas dados do usuário.

A opção pelo BFF se justifica pela segurança. Os três segredos de runtime (`JWT_SECRET`, `INTERNAL_API_KEY` e `TMDb_API_TOKEN`) ficam no servidor, nunca no navegador. O cliente só conhece rotas same-origin, e o token de sessão circula em cookie `HttpOnly`. O salto interno Next para Nest é o único que carrega a chave compartilhada e o Bearer JWT, de modo que nenhuma credencial chega ao JavaScript da página.

Para reduzir latência e consumo da API externa, a API mantém em cache os resultados da TMDb, com prazos calibrados pela frequência com que cada informação muda: busca, descoberta e gêneros ficam em cache por uma hora; detalhes da ficha, mais estáveis, por 24 horas; e os provedores de onde assistir, que variam com contratos de licenciamento, por 12 horas. Na infraestrutura, o frontend é publicado na Vercel e a API roda em uma máquina virtual da Azure na região Brazil South, atrás do servidor Caddy, responsável por terminar o TLS e servir a aplicação em HTTPS.

3.2 Banco de dados

Figura 2 – Diagrama entidade-relacionamento do Plot Twist



Fonte: elaborado pelos autores (2026).

O modelo persistente é propositalmente enxuto, como mostra a Figura 2. Ele tem quatro entidades dedicadas à conta e à atividade do usuário: User, Rating, Watched e Favorite. Existem três relações um-para-muitos, todas partindo de User: um usuário pode ter muitas avaliações, muitas marcações de visto e muitos favoritos.

As três entidades de atividade compartilham uma restrição de unicidade composta (`userId`, `tmdbId`, `tmdbType`). Assim, cada usuário mantém no máximo um registro de cada tipo para o mesmo título. O par (`tmdbId`, `tmdbType`) identifica o título na TMDB e distingue filmes (MOVIE) de séries (TV); a unicidade real inclui `userId`, e esse par não é chave estrangeira para uma tabela local.

Os títulos não são persistidos no banco. Sinopse, elenco, imagens, gêneros e provedores vêm da TMDB a cada consulta, com cache. Persistir o catálogo seria redundante, traria o custo de manter esses dados sincronizados com a fonte externa e não agregaria valor ao MVP. O banco guarda só o que é próprio do usuário.

O schema usa dois enums no nível do banco: `TmdbType`, com os valores MOVIE e TV, e `WatchedOrigin`, exclusivo de `Watched.origem`, com manual e auto. O campo `Rating.score` é do tipo Float, embora o DTO atual aceite apenas inteiros de 1 a 10; a escolha deixa espaço para avaliações fracionárias futuras sem exigir uma migration de tipo. A estrutura foi entregue em duas migrations versionadas (20260523123455_init e

20260618122517_history_enum_and_origem), o que dá rastreabilidade à evolução do banco desde a criação inicial até a introdução do enum de origem do visto.

3.3 Backend

O backend é uma aplicação NestJS 11 organizada em módulos por domínio, cada um encapsulando controller, service e regras próprias:

- **auth:** cadastro, login e identificação do usuário logado. A senha é armazenada com hash bcrypt (fator de custo 10). O login emite o JWT de 24 horas, é protegido por rate-limit de cinco tentativas a cada 15 minutos por IP e responde com a mesma mensagem genérica tanto para email inexistente quanto para senha incorreta, para não revelar quais emails têm conta;
- **titles:** busca, descoberta (por gênero ou populares), ficha do título com provedores do Brasil e listagem de gêneros. A ficha aceita o JWT de forma opcional: quando presente, a resposta inclui o estado do usuário para aquele título;
- **history:** avaliação, visto e favorito, além do perfil do usuário. Todas as rotas exigem JWT.

As regras de negócio do histórico merecem destaque por encadearem as três atividades. Avaliar um título marca-o como visto automaticamente, com origem auto. Remover a avaliação derruba o visto apenas se a origem for auto; se o usuário tinha marcado o visto manualmente, ele permanece. Na direção oposta, marcar manualmente como visto um título que já estava visto por origem auto promove o registro para manual — a partir daí ele sobrevive à remoção da avaliação. E não é permitido desmarcar como visto um título que ainda tem avaliação ativa: nesse caso a API responde com conflito HTTP 409, comunicando ao cliente que a operação viola uma regra de negócio. As marcações usam upsert e as remoções usam deleteMany, o que torna as operações idempotentes: repetir a mesma requisição produz o mesmo estado final, sem erros nem duplicidades.

A validação das entradas usa o class-validator com ValidationPipe global, que remove campos não permitidos e transforma os dados. A segurança de borda combina dois guards: o InternalKeyGuard global exige o header X-Internal-Key em todas as rotas, com exceção pública para /health; e o JwtAuthGuard (ou, na ficha, o OptionalJwtGuard, que só preenche o estado do usuário quando há token) protege as rotas de histórico e perfil. O rate-limit global de 100 requisições por minuto por IP é aplicado pelo XClientIpThrottlerGuard, que lê o header X-Client-IP propagado pelo BFF — necessário porque, do ponto de vista do Nest, todas as

requisições chegam do servidor do Next, e sem essa propagação o limite puniria todos os usuários como se fossem um só. Helmet aplica cabeçalhos de segurança e os logs têm redação de dados sensíveis.

3.4 Frontend

O frontend é uma aplicação Next.js 16 com React 19, usando o App Router. São seis telas: Home (/), busca (/busca), ficha do título (/titulo/[type]/[id]), login (/login), cadastro (/cadastro) e perfil (/perfil, protegida). A navegação parte da Home, que concentra a busca, a grade de populares (“Em alta”) e os chips de gênero, e leva à ficha de cada título.

A camada de Route Handlers em `src/app/api/*` é o BFF e espelha um para um os endpoints da API NestJS. É essa camada que lê o cookie de sessão e injeta os headers internos nas chamadas ao backend, mantendo as credenciais fora do navegador.

Os componentes principais são o card de pôster (usado na busca e na descoberta), a ficha do título, o bloco de “onde assistir” com os provedores separados por assinatura, aluguel e compra, e os controles de avaliação, visto e favorito. A identidade visual “Plot Twist” usa Tailwind v4 com cores em OKLCH, tem o tema escuro como padrão (alternado e persistido com a biblioteca next-themes) e um tema claro em tons quentes, e adota a fonte Inter na interface, reservando a Fraunces para a marca. A interface é responsiva, com piso de funcionamento garantido em telas a partir de 360 pixels.

O gerenciamento de estado assíncrono usa TanStack Query, que cuida de cache, revalidação e estados de carregamento das chamadas ao BFF, e os formulários (login e cadastro) usam TanStack Form com validação via Zod, garantindo que as mesmas regras de formato sejam verificadas antes mesmo de a requisição sair do navegador. O React Compiler está habilitado no build, memoizando componentes automaticamente sem exigir `useMemo/useCallback` manuais. Durante o desenvolvimento paralelo das duas frentes, um conjunto de mocks por módulo (`*.mock.ts`) permitiu rodar o frontend isolado do backend antes da integração, o que destravou o trabalho simultâneo das equipes.

3.5 API REST

A API REST do NestJS é o contrato de domínio do sistema. Em cada rota, o navegador chama o espelho no BFF (`/api/...` no Next), que repassa à rota correspondente do Nest. Todas as rotas, exceto `/health`, exigem o header `X-Internal-Key`; as rotas de histórico e perfil exigem

também o JWT. O Quadro 1 resume os endpoints; a referência completa, com exemplos de requisição e resposta, está em docs/api.md nos repositórios.

Quadro 1 – Endpoints da API REST

Método e rota (Nest)	Autenticação	Função
GET /health	pública	Verificação de saúde da API
POST /auth/register	chave interna	Cadastro de usuário
POST /auth/login	chave interna + rate-limit	Login, emite JWT de 24h
GET /auth/me	chave interna + JWT	Dados do usuário logado
GET /titles/search?q&page	chave interna	Busca de filmes e séries
GET /titles/discover?genre?&page	chave interna	Descoberta por gênero ou populares
GET /titles/:type/:id	chave interna + JWT opcional	Ficha do título com provedores BR
GET /genres	chave interna	Lista de gêneros
POST / DELETE /titles/:type/:id/rating	JWT	Avaliar e remover avaliação
POST / DELETE /titles/:type/:id/watched	JWT	Marcar e desmarcar visto
POST / DELETE /titles/:type/:id/favorite	JWT	Favoritar e desfavoritar
GET /users/me/profile	JWT	Totais e listas do perfil

Fonte: elaborado pelos autores (2026).

O logout não tem endpoint no Nest: é resolvido só no BFF (POST /api/auth/logout), que apaga o cookie de sessão. Além do contrato funcional, existem rotas internas de suporte (/user/*), sem espelho no BFF e acessíveis apenas com a chave interna; elas estão anotadas em docs/api.md.

4 DOCUMENTAÇÃO DAS TELAS

As seis telas do sistema estão descritas a seguir. Para cada uma, apresentam-se o objetivo, o fluxo, as ações disponíveis e as capturas da aplicação publicada em desktop, em tema escuro e em tema claro. O mesmo catálogo de capturas está versionado em docs/telas/ nos repositórios do projeto.

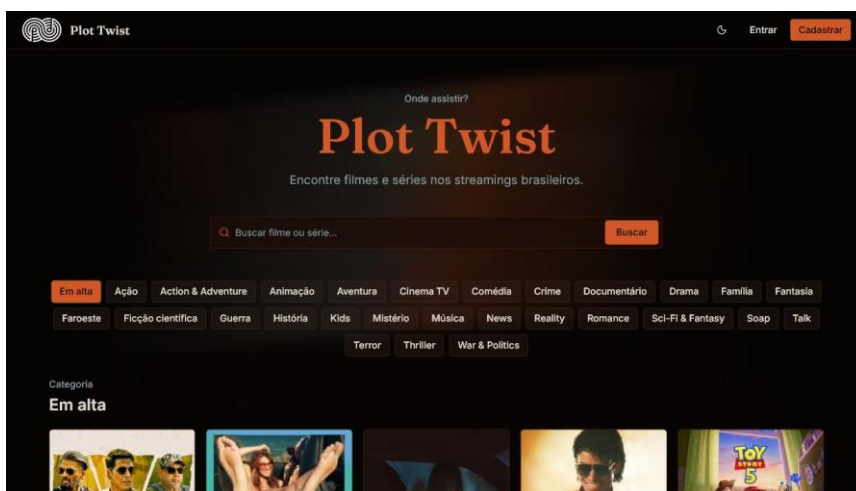
4.1 Home

Objetivo: porta de entrada para buscar ou descobrir títulos.

Fluxo: hero com a marca “Plot Twist” e a busca; abaixo, grade “Em alta” e chips de gênero; clicar num card abre a ficha.

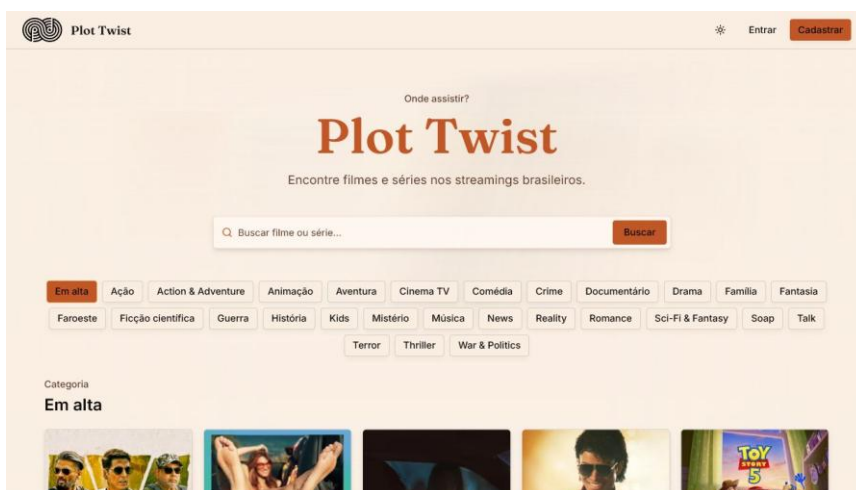
Ações: buscar por texto; filtrar por gênero; abrir a ficha de um título; alternar tema claro e escuro; entrar ou cadastrar.

Figura 3 – Tela Home em tema escuro



Fonte: elaborado pelos autores (2026).

Figura 4 – Tela Home em tema claro



Fonte: elaborado pelos autores (2026).

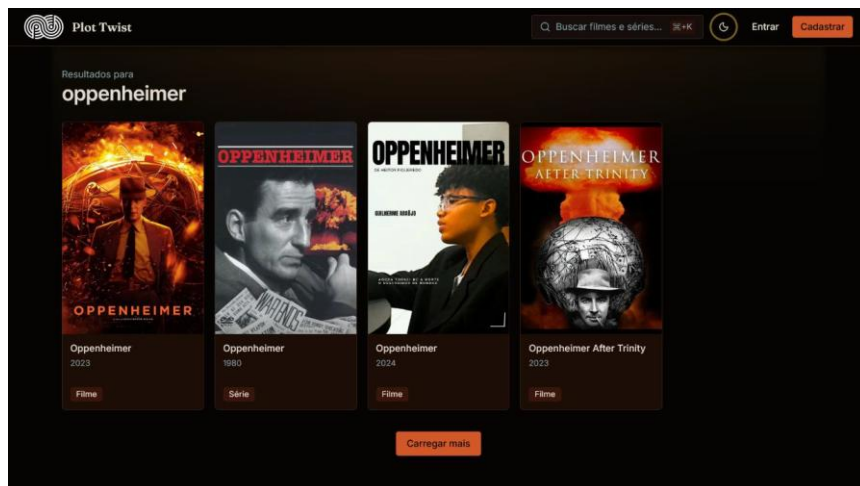
4.2 Busca

Objetivo: listar resultados de filmes e séries para um termo.

Fluxo: o termo vem da Home ou do header; a grade mostra cards com pôster, título, ano e selo Filme/Série; sem termo, volta para a Home.

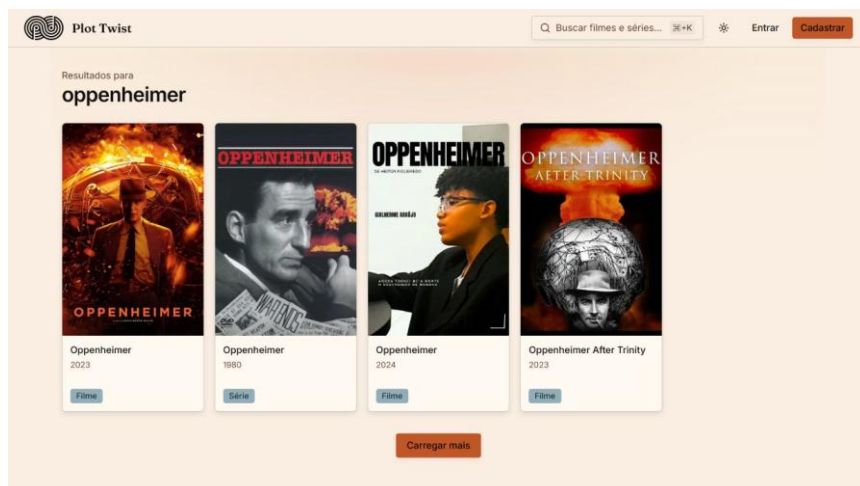
Ações: abrir ficha; refazer a busca; favoritar direto no card, quando logado.

Figura 5 – Tela Busca em tema escuro



Fonte: elaborado pelos autores (2026).

Figura 6 – Tela Busca em tema claro



Fonte: elaborado pelos autores (2026).

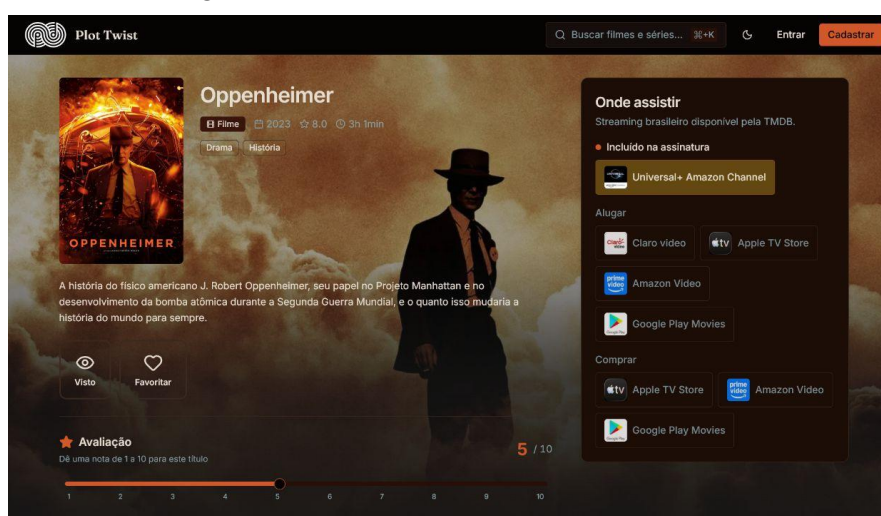
4.3 Ficha do título

Objetivo: detalhes completos e onde assistir.

Fluxo: pôster e backdrop, sinopse, elenco, nota da TMDb, gêneros, duração ou temporadas; bloco destacado “Onde assistir” (assinatura, aluguel, compra) com provedores do Brasil; controles de avaliação, visto e favorito.

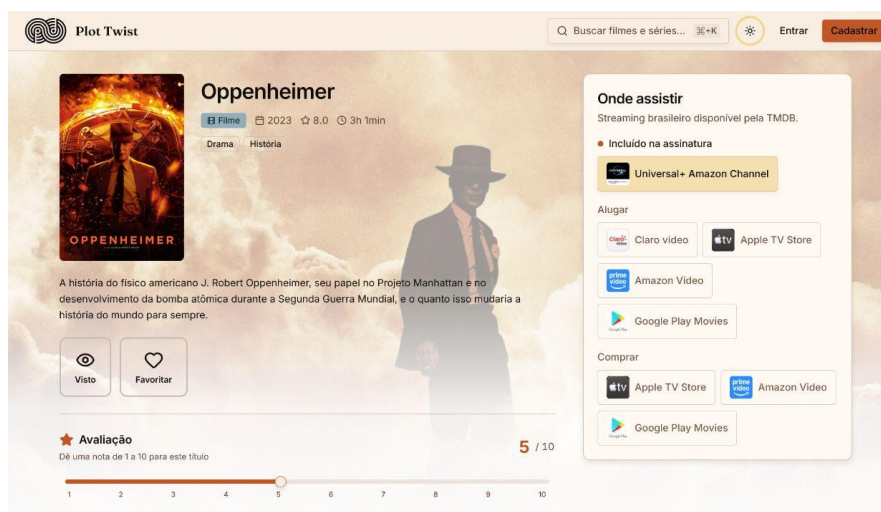
Ações: avaliar de 1 a 10; marcar visto; favoritar; ver provedores. As ações de histórico pedem login.

Figura 7 – Tela Ficha do título em tema escuro



Fonte: elaborado pelos autores (2026).

Figura 8 – Tela Ficha do título em tema claro



Fonte: elaborado pelos autores (2026).

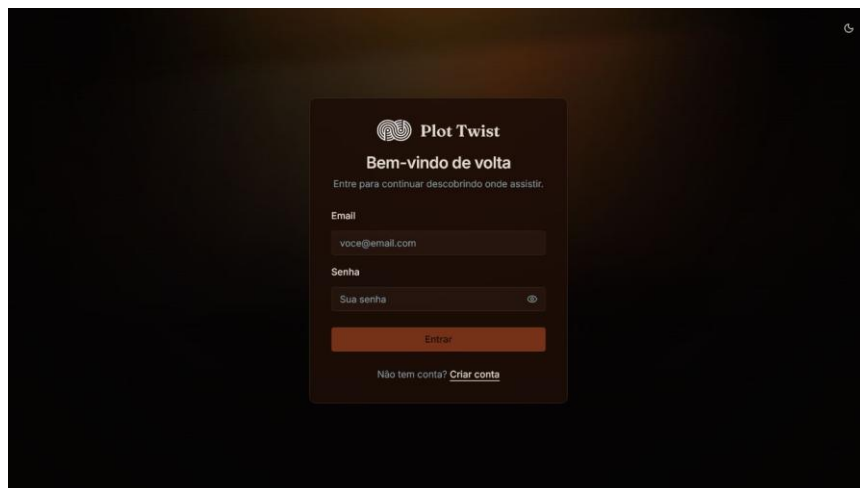
4.4 Login

Objetivo: autenticar usuário existente.

Fluxo: email e senha; o sucesso cria a sessão por cookie e devolve o usuário à navegação; mostra aviso quando o acesso vem do cadastro.

Ações: entrar; ir para o cadastro.

Figura 9 – Tela Login em tema escuro



Fonte: elaborado pelos autores (2026).

Figura 10 – Tela Login em tema claro



Fonte: elaborado pelos autores (2026).

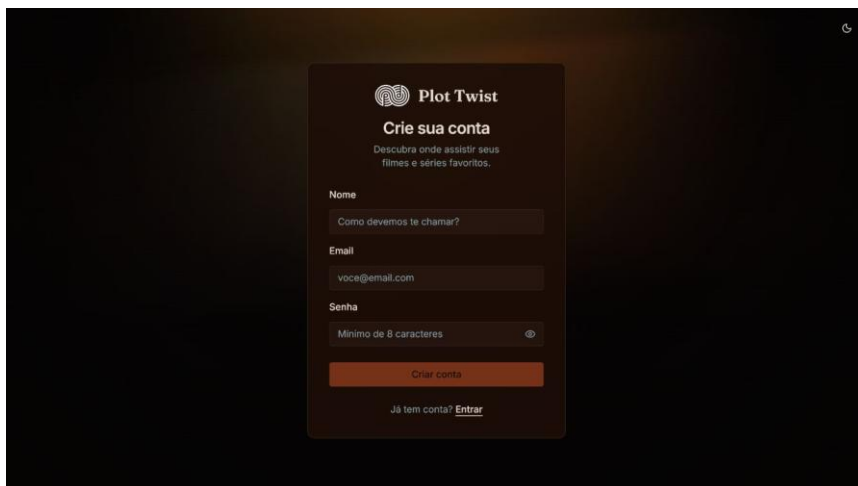
4.5 Cadastro

Objetivo: criar conta.

Fluxo: nome (3 a 50 caracteres), email e senha (mínimo 8); ao concluir, o usuário segue para o login, já que o cadastro não inicia sessão.

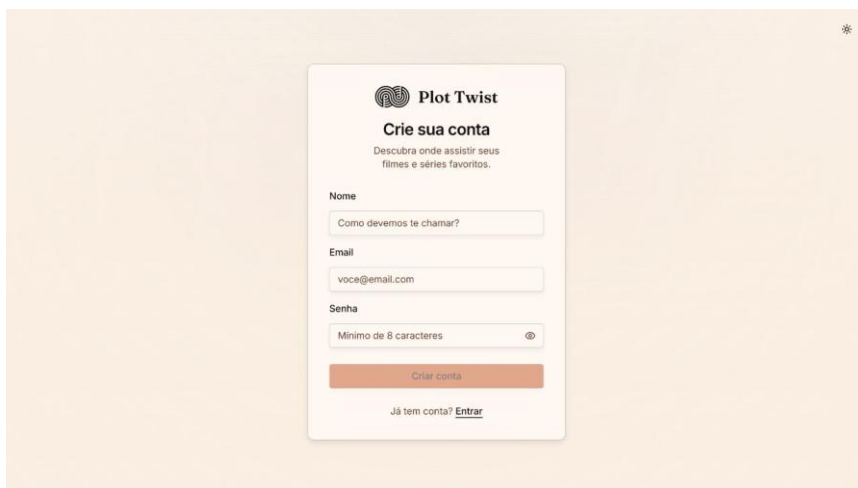
Ações: cadastrar; ir para o login.

Figura 11 – Tela Cadastro em tema escuro

A imagem mostra a tela de cadastro do aplicativo Plot Twist em um tema escuro. O formulário centralizado contém o logo da Plot Twist, o título "Crie sua conta", o subtítulo "Descubra onde assistir seus filmes e séries favoritos.", e campos de entrada para "Nome" (com o placeholder "Como devemos te chamar?"), "Email" (com o placeholder "voce@email.com") e "Senha" (com o placeholder "Mínimo de 8 caracteres" e um ícone de olho para alternar visibilidade). Abaixo dos campos, há um botão laranja "Criar conta" e um link "Já tem conta? Entrar" em azul.

Fonte: elaborado pelos autores (2026).

Figura 12 – Tela Cadastro em tema claro

A imagem mostra a tela de cadastro do aplicativo Plot Twist em um tema claro. O formulário centralizado contém o logo da Plot Twist, o título "Crie sua conta", o subtítulo "Descubra onde assistir seus filmes e séries favoritos.", e campos de entrada para "Nome" (com o placeholder "Como devemos te chamar?"), "Email" (com o placeholder "voce@email.com") e "Senha" (com o placeholder "Mínimo de 8 caracteres" e um ícone de olho para alternar visibilidade). Abaixo dos campos, há um botão laranja "Criar conta" e um link "Já tem conta? Entrar" em azul.

Fonte: elaborado pelos autores (2026).

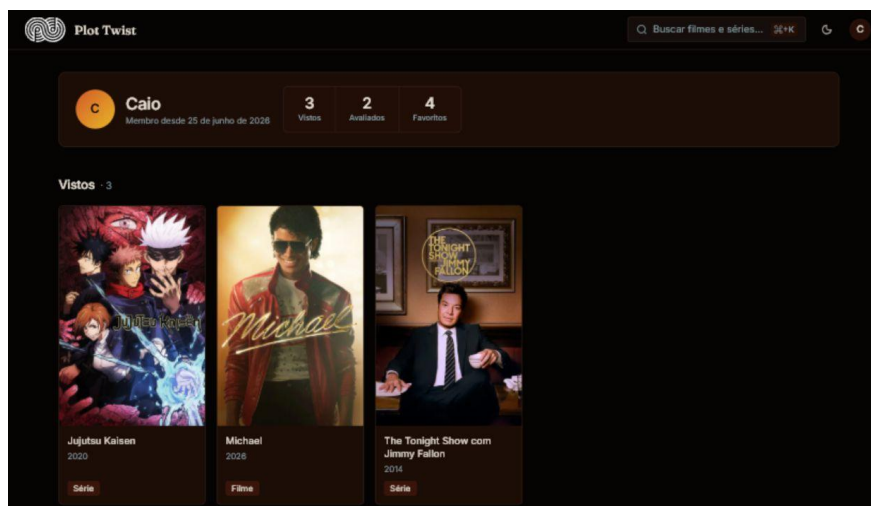
4.6 Perfil

Objetivo: painel pessoal do usuário logado.

Fluxo: cabeçalho com o nome; totais de vistos, avaliados e favoritos; listas recentes de cada categoria. Sem sessão, a tela redireciona para o login.

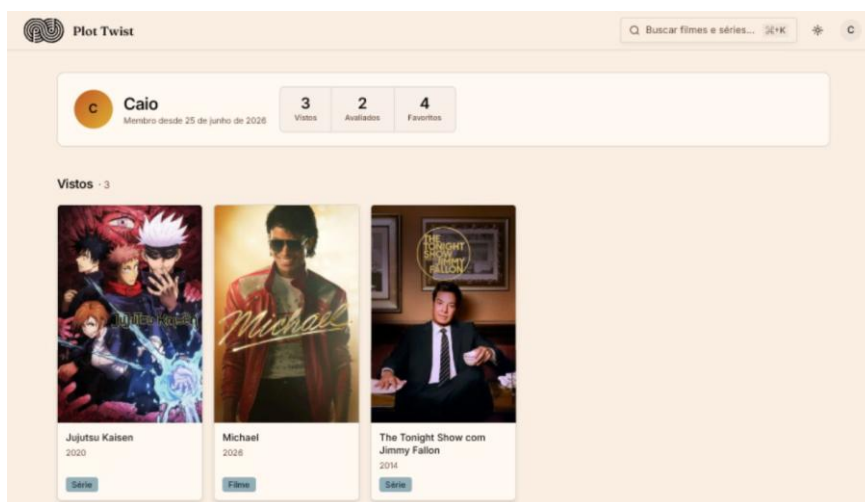
Ações: navegar para fichas a partir das listas; sair (logout).

Figura 13 – Tela Perfil em tema escuro



Fonte: elaborado pelos autores (2026).

Figura 14 – Tela Perfil em tema claro



Fonte: elaborado pelos autores (2026).

5 ORGANIZAÇÃO ÁGIL E BACKLOG

O projeto adotou Scrum enxuto. O backlog viveu nas issues do GitHub dos dois repositórios, organizadas por epic (via labels) e por sprint (via milestones). Cada item foi entregue por feature branch, integrado por Pull Request com revisão do time, e mergeado na main. A síntese navegável do backlog está em docs/backlog.md.

A equipe foi composta por doze integrantes, com papéis distribuídos conforme o Quadro 2. A divisão por especialidade (backend, frontend, QA e documentação) permitiu paralelizar as frentes de trabalho, enquanto o papel de Product Owner concentrou a priorização do backlog.

Quadro 2 – Integrantes da equipe e papéis

Nome	RA	Função
Caio Parada Oliveira Planinschek	1240205596	Product Owner, Backend, Frontend
João Victor Berçot Chabudet Cabral	1240108001	Backend
Igor Rocha Lobato	1240114118	Backend, Frontend
Carlos Matheus Marinho Tavares	1240119803	Frontend, Documentação
Eduardo de Oliveira Fernandes	1240104170	Backend
Luiz Felipe de Aguiar Passaroni	1240114559	QA
Lucas Vasconcelos Tabosa de Almeida	1250100134	Documentação
Gabriel Albuquerque Varela Santarelli	1240110815	Documentação, QA
Davi Hasson Castro	1240117220	Frontend
Maria Fernanda Lourenço Class Teixeira	1240113501	Documentação, QA
Gabriel Silva Franciscone	1240117399	Documentação
Felipe Valentim Gabry	1240114958	Documentação

Fonte: elaborado pelos autores (2026).

Ao todo foram 66 issues: 39 no backend (37 fechadas e 2 abertas) e 27 no frontend (todas fechadas). As duas issues ainda abertas no backend são de documentação de entrega (epic meta) e correspondem à própria entrega final: o relatório acadêmico e o empacotamento do código-fonte.

Epics. O trabalho se dividiu em oito epics: auth (cadastro, login, logout e sessão), catalogo (busca, ficha, onde assistir, gêneros e discover), historico (avaliar, visto, favorito e perfil), infra (setup, env, BFF, guards, rate-limit e segurança), testes (cenários de teste obrigatórios do PRD, transversal aos dois repos), ui (tema claro/escuro e responsividade, no front), deploy (publicação online e HTTPS) e meta (kickoff, documentação e entrega acadêmica).

Sprints. A entrega foi marcada por quatro milestones, cada um com um objetivo claro de incremento:

- **Sprint 0 (Setup):** preparar o terreno, com os dois repositórios rodando localmente e CI/CD de pé, sem demo funcional ainda;
- **Sprint 1 (Núcleo demonstrável):** auth com BFF, busca, ficha e onde assistir, rodando localmente;
- **Sprint 2 (Histórico, perfil, UI e deploy):** avaliar, visto e favorito, perfil, Home, tema, responsividade e publicação online;
- **Fechamento (Documentação de entrega):** os artefatos acadêmicos da entrega final, sob o epic meta.

A lista completa de issues abertas e fechadas, com os respectivos milestones, está disponível nos dois repositórios e serve de evidência da organização ágil do time.

6 VERSIONAMENTO E GITHUB

O código vive em dois repositórios no GitHub:

- **Backend:** <https://github.com/luizpassaroni/lab-dev-software-back>
- **Frontend:** <https://github.com/luizpassaroni/lab-dev-software-front>

O fluxo de versionamento foi baseado em feature branches. Cada funcionalidade nasceu em sua própria branch, foi aberta como Pull Request e passou por revisão de pares antes de ser integrada à main (na Sprint 2, parte do trabalho passou por uma branch develop antes da main). Esse fluxo garantiu que nenhuma mudança chegasse à linha principal sem ao menos um segundo olhar, elevando a qualidade e distribuindo o conhecimento do código pelo time. As mensagens de commit seguiram a convenção Conventional Commits, com os prefixos feat, fix, docs, test, chore e refactor — por exemplo, feat: criação da tela de login ou fix: correção da validação de usuário —, o que torna o histórico legível e pesquisável.

A integração contínua e a entrega contínua foram automatizadas com GitHub Actions. No backend, o fluxo aplica a infraestrutura declarada em Bicep, constrói a imagem Docker, publica-a no GitHub Container Registry (GHCR), conecta-se à VM da Azure por SSH, sobe os serviços com Docker Compose (Caddy, NestJS e PostgreSQL) e executa prisma migrate deploy ao final, garantindo que o banco de produção esteja sempre alinhado ao schema versionado. O frontend é publicado na Vercel, que gera um deploy automático a cada integração na main.

7 RESULTADOS

O sistema está no ar e funcional. O frontend está publicado na Vercel em <https://lab-dev-software-front.vercel.app> e a API responde por HTTPS em <https://api-uva.eduoncode.com>, com a VM hospedada na região Brazil South da Azure e o PostgreSQL fora do alcance da internet pública.

O Quadro 3, verificado em 27/06/2026, comprova o isolamento do backend imposto pelo BFF: o acesso direto ao Nest sem a chave interna é rejeitado, e o navegador só consegue chegar à API pelas rotas espelho do Next.

Quadro 3 – Verificações do ambiente publicado (27/06/2026)

Verificação	Resultado
Frontend (Vercel)	https://lab-dev-software-front.vercel.app – HTTP 200
API HTTPS /health	https://api-uva.eduoncode.com/health – HTTP 200 {"status": "ok"}
Redirecionamento HTTP → HTTPS	http://api-uva.eduoncode.com/health – HTTP 308
BFF /api/genres	https://lab-dev-software-front.vercel.app/api/genres – HTTP 200
Nest /genres sem chave interna	https://api-uva.eduoncode.com/genres – HTTP 401 (esperado)

Fonte: elaborado pelos autores (2026).

A qualidade foi sustentada por testes automatizados: no backend, 19 suítes unitárias (99 casos, com Jest) e 7 suítes end-to-end (test/*.e2e-spec.ts, cerca de 50 casos); no frontend, 5 arquivos de teste (10 casos, com Vitest). No backend, o gate npx prisma generate precede o build e os testes, evitando que o cliente do ORM fique defasado em relação ao schema.

A validação ponta a ponta foi executada com sucesso sobre o ambiente publicado, percorrendo o fluxo completo sem falhas: cadastro (que já realiza o login automático e leva à Home), persistência de sessão após recarregar a página, busca de filme e de série com o selo correto, abertura da ficha com a informação de onde assistir e os provedores do Brasil, avaliação de nota 8 com marcação automática de visto, confirmação da regra de conflito 409 ao tentar desmarcar um visto com avaliação ativa, perfil refletindo as ações (um visto, um avaliado e um favorito), logout com o devido redirecionamento da página de perfil para o login, e alternância de tema. Em telas de 360 pixels o layout permanece utilizável, com um único ponto cosmético a refinar: um leve deslocamento horizontal originado no cabeçalho, que não afeta o uso.

Em relação ao planejado, o MVP entregou todo o escopo previsto: cadastro e login com sessão segura, busca de filmes e séries, ficha com onde assistir, descoberta por gênero e

populares, avaliação, visto, favorito, perfil e tema claro/escuro responsivo. O custo da infraestrutura Azure foi acompanhado e ficou dentro do esperado.

8 CONSIDERAÇÕES FINAIS

O Plot Twist cumpriu o objetivo proposto: entregar, em produção, um sistema full-stack que resolve de forma direta a pergunta de onde assistir a um filme ou série no Brasil. O time exercitou o ciclo completo de desenvolvimento, da modelagem ao deploy, com versionamento colaborativo e processo ágil, e os objetivos específicos definidos na introdução foram todos atendidos pelo escopo entregue.

As principais dificuldades foram três. A primeira foi coordenar uma equipe grande com disponibilidade desigual, o que exigiu disciplina de backlog e de revisão por Pull Request para manter o ritmo. A segunda foi integrar uma API externa: a TMDB define o formato dos dados, impõe limites de uso e exige cuidado com idioma e região, o que motivou a camada de cache e a normalização das respostas. A terceira foi o deploy com HTTPS, que envolveu provisionar a VM da Azure, configurar o Caddy para terminar TLS e orquestrar os contêineres com Docker Compose no pipeline.

Entre os aprendizados, destacam-se o valor do padrão BFF para manter segredos fora do cliente, a clareza que migrations e um modelo de dados enxuto trazem para o histórico do usuário, e o ganho de produtividade de uma esteira de CI/CD bem definida. No plano humano, a experiência de dividir doze pessoas em frentes paralelas, sincronizadas por issues, milestones e Pull Requests, mostrou na prática por que processo e ferramenta de versionamento são tão decisivos quanto o código em si.

Como melhorias futuras, o produto comporta uma lista de “quero assistir” (watchlist), um motor de recomendação baseado no histórico do usuário e uma blocklist de tokens para invalidar sessões antes do prazo de expiração. Nenhuma dessas evoluções exige mudança estrutural: a arquitetura em camadas e o modelo de dados enxuto foram desenhados para absorvê-las de forma incremental.

REFERÊNCIAS

- BECK, Kent et al. **Manifesto para o Desenvolvimento Ágil de Software**. 2001. Disponível em: <https://agilemanifesto.org/iso/ptbr/manifesto.html>. Acesso em: 30 jun. 2026.
- CHACON, Scott; STRAUB, Ben. **Pro Git**. 2. ed. Apress, 2014. Disponível em: <https://git-scm.com/book/pt-br/v2>. Acesso em: 30 jun. 2026.
- FIELDING, Roy Thomas. **Architectural Styles and the Design of Network-based Software Architectures**. Tese (Doutorado) — University of California, Irvine, 2000. Disponível em: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>. Acesso em: 30 jun. 2026.
- JONES, M.; BRADLEY, J.; SAKIMURA, N. **JSON Web Token (JWT)**. RFC 7519. IETF, maio 2015. Disponível em: <https://datatracker.ietf.org/doc/html/rfc7519>. Acesso em: 30 jun. 2026.
- NESTJS. **NestJS Documentation**. 2026. Disponível em: <https://docs.nestjs.com>. Acesso em: 30 jun. 2026.
- NEWMAN, Sam. **Building Microservices: Designing Fine-Grained Systems**. 1. ed. Sebastopol: O'Reilly Media, 2015.
- NEXT.JS. **Next.js Documentation**. Vercel, 2026. Disponível em: <https://nextjs.org/docs>. Acesso em: 30 jun. 2026.
- POSTGRESQL GLOBAL DEVELOPMENT GROUP. **PostgreSQL Documentation**. 2026. Disponível em: <https://www.postgresql.org/docs>. Acesso em: 30 jun. 2026.
- PRISMA. **Prisma ORM Documentation**. Prisma Data, Inc., 2026. Disponível em: <https://www.prisma.io/docs>. Acesso em: 30 jun. 2026.
- REACT. **React Documentation**. Meta Open Source, 2026. Disponível em: <https://react.dev>. Acesso em: 30 jun. 2026.
- SCHWABER, Ken; SUTHERLAND, Jeff. **The Scrum Guide: The Definitive Guide to Scrum**. 2020. Disponível em: <https://scrumguides.org>. Acesso em: 30 jun. 2026.
- THE MOVIE DATABASE. **TMDB API Documentation**. 2026. Disponível em: <https://developer.themoviedb.org/docs>. Acesso em: 30 jun. 2026.